

Full Length Article

Blinding and blurring the multi-object tracker with adversarial perturbations

Haibo Pang^a, Rongqi Ma^a, Jie Su^{a,*}, Chengming Liu^a, Yufei Gao^a, Qun Jin^b^a School of Cyber Science and Engineering, Zhengzhou University, Zhengzhou City, 450003, China^b Waseda University, Tokorozawa, 359-1192, Japan

ARTICLE INFO

Keywords:

Multi-object tracking
Adversarial attack
Object detection
Computer vision

ABSTRACT

Adversarial attack reveals a potential imperfection in deep models that they are susceptible to being tricked by imperceptible perturbations added to images. Recent deep multi-object trackers combine the functionalities of detection and association, rendering attacks on either the detector or the association component an effective means of deception. Existing attacks focus on increasing the frequency of ID switching, which greatly damages tracking stability, but is not enough to make the tracker completely ineffective. To fully explore the potential of adversarial attacks, we propose Blind-Blur Attack (BBA), a novel attack method based on spatio-temporal motion information to fool multi-object trackers. Specifically, a simple but efficient perturbation generator is trained with the blind-blur loss, simultaneously making the target invisible to the tracker and letting the background be regarded as moving targets. We take TraDeS as our main research tracker, and verify our attack method on other excellent algorithms (*i.e.*, CenterTrack, FairMOT, and ByteTrack) on MOT-Challenge benchmark datasets (*i.e.*, MOT16, MOT17, and MOT20). BBA attack reduced the MOTA of TraDeS and ByteTrack from 69.1 and 80.3 to -238.1 and -357.0, respectively, indicating that it is an efficient method with a high degrees of transferability.

1. Introduction

Multi-object tracking (MOT) is a long-standing problem of tracking the unknown number of targets with important applications such as autonomous driving, video surveillance, mobile robotics, and others. Unlike object detection and the Re-identification (ReID) system, MOT aims to predict the trajectories of multiple objects in a video sequence and label each of them with a fixed ID. Recent trackers generally adopt a strategy of combining deep object detector with ReID systems and achieve leading performance on popular benchmarks, which can be roughly classified as tracking-by-detection (TBD) (Bewley, Ge, Ott, Ramos, & Upcroft, 2016; Rakai, Song, Sun, Zhang, & Yang, 2022; Shuai, Berneshawi, Li, Modolo, & Tighe, 2021; Wojke, Bewley, & Paulus, 2017; Xu, Cao, Zhang, & Hu, 2019; Zhang et al., 2021) and joint-detection-tracking (JDT) (Guo, Wang, Wang, & Tao, 2021; Lu, Rathod, Votel, & Huang, 2020; Wang, Zheng, Liu, Li, & Wang, 2020; Wang, Zheng, Pan, & Xu, 2021; Zhang, Cheng, Zhu, Lin, & Dai, 2018; Zhang, Wang, Wang, Zeng, & Liu, 2020; Zhou, Koltun, & Krhenbühl, 2020). The former divides the tracking problem into two independent subtasks: object detection and data association. The detection task is to identify objects independently in every frame, and the association component is responsible for combining all detection boxes output by detector. With the help of excellent detectors such as Faster-RCNN (Ren, He, Girshick,

& Sun, 2017), SSD (Liu et al., 2016), and YOLO (Ge, Liu, Wang, Li, & Sun, 2021; Redmon & Farhadi, 2018; YOLOv5, 2020), the performance of TBD-based trackers is greatly improved.

On the contrary, JDT-based trackers regard MOT as a single-stage problem and adopt a shared backbone network for joint detection and tracking. This approach enables joint training of object detection and association component, greatly enhancing their performance. In recent years, the JDT paradigm has attracted increasing attention from online trackers due to its real-time advantages.

Adversarial attack, as demonstrated by Szegedy, Zaremba, Sutskever, Bruna, Erhan, Goodfellow, and Fergus (2013), is an effective method to fool deep learning models by introducing subtle adversarial noise. The research about it helps people understand the working principle of Deep Neural Networks (DNNs) and facilitates building robust models. Popular adversarial attack methods have been used extensively in image classification (Duan et al., 2021; He, Zhang, Ren, & Jian, 2016; Laykaviriyakul & Phaisangittisagul, 2023; Simonyan & Zisserman, 2014), object detection (Wang, Liu, Yin, Liu, & Liu, 2021; Wei, Liang, Chen, & Cao, 2018), semantic segmentation (Chen, Papan-dreou, Kokkinos, Murphy, & Yuille, 2018; Fischer, Kumar, Metzen, & Brox, 2017; Nakka & Salzmänn, 2019), and single object tracking (Guo,

* Corresponding author.

E-mail addresses: hbpang@zzu.edu.cn (H. Pang), rongmiq@gmail.com (R. Ma), sujie@gs.zzu.edu.cn (J. Su), cmliu@zzu.edu.cn (C. Liu), yfgao@zzu.edu.cn (Y. Gao), jinq@waseda.jp (Q. Jin).<https://doi.org/10.1016/j.neunet.2024.106331>

Received 30 November 2023; Received in revised form 18 March 2024; Accepted 21 April 2024

Available online 25 April 2024

0893-6080/© 2024 Elsevier Ltd. All rights reserved.

Cheng, Juefei-Xu, Ma, & Zhao, 2021; Jia, Song, Ma, & Yang, 2021; Liang, Wei, Yao, & Cao, 2020; Yan, Wang, Lu, & Yang, 2020). Generally speaking, adversarial attacks can be categorized into white-box attacks, where attackers have complete knowledge of the model (e.g., network structure, parameters, and training data), and black-box attacks, where attackers have limited information. Additionally, attacks can also be categorized based on the environment: digital world attacks occur primarily in the digital space by modifying image data, while physical world attacks focus on the visual characteristics of real-world objects. In traditional works (Goodfellow, Shlens, & Szegedy, 2014; Kurakin, Goodfellow, & Bengio, 2016; Moosavi-Dezfooli, Fawzi, & Frossard, 2016), adversarial examples usually need to go through multiple iterations of the gradient rise algorithm to obtain. These methods change the most critical pixels in the image, which is helpful but time-consuming. Current methods (Wang, Wang, Li, Zhang, & Lin, 2020; Xiao et al., 2018; Xie, & Tao, 2019; Yan et al., 2020) adopt deep networks as noise generator to produce adversarial examples efficiently. Generative Adversarial Networks (GAN) (Goodfellow, Pouget-Abadie, et al., 2014) is a typical representation of these networks, which has received extensive attention in the field of adversarial attack.

While previous works (Guo, Cheng, et al., 2021; Wang, Liu, et al., 2021; Wei et al., 2018; Yan et al., 2020) have successfully attacked detectors and single object trackers, an efficient and transferable attack method for multi-object trackers remains elusive. Hijacking (Jia et al., 2019) is the first proposed and rare method for attacking MOT. It generates fabricated adversarial bounding boxes by adding adversarial patches to video frames, effectively erasing the original detection bounding boxes. In this way, the detected object bounding box can no longer be associated with any specific tracking trajectory, causing MOT to fail in autonomous driving. The goal of Hijacking is to attack some specified tracking trajectories, which are effective but time-consuming for TBD trackers with Kalman filters. To achieve effective and efficient attacks, F&F attack (Zhou et al., 2023) introduces a novel method that combines false alarms, false negatives, and association attacks to deceive crowded scenes, misleading the tracker's detection and association component. Through three designs, it enhances deception and erases detections to further mislead association. The ultimate aim is to compromise the entire tracking system by fooling its detection, thus improving attack effectiveness.

Despite achieving successful outcomes through the above methods, there are still two notable issues: 1. The considerable number of ID switches resulting from attacks can significantly against the tracking process, but it did not completely crush MOT trackers. 2. The attack methods targeting detection results or association rules lack complete transferability, as they do not directly disrupt the unique motion modeling ability of the tracker.

To achieve more comprehensive and diversified attacks, we propose an efficient and transferable method called BBA to blind and blur multi-object trackers including JDT-based and TBD-based. Our attack method aims to achieve two objectives: 1. Blinding tracker, preventing it from seeing foreground objects. 2. Blurring tracker, making it fail to distinguish between the objects and the background. To achieve these, we introduce an adversarial loss function named blind-blur loss, which implicitly against the detector and association component by disrupting their object recognition and motion prediction capabilities. In this paper, the blind-blur loss serves as a guide for noise generators, enabling them to produce effective adversarial examples. Different from previous works (Jia et al., 2019; Zhou et al., 2023), our attack is not limited to attacking specific trajectories, but is an indiscriminate and strongly adaptive attack method. We choose TraDeS (Wu, Cao, Song, Wang, & Yuan, 2021) as our primary attacked tracker, it is JDT-based tracker which treats tracking task as a motion estimation problem. Our attack can also be used as a black-box method against other trackers such as CenterTrack (Zhou et al., 2020), FairMOT (Zhang et al., 2020), and ByteTrack (Zhang et al., 2021). In addition, our attack method takes less than 3 ms to generate a high-quality example on single GPU, which makes the attack hard to be detected. In summary, our main contributions can be listed below:

- An efficient and general method to generate adversarial examples against multi-object trackers is proposed. Extensive experiments indicate that our attack method can pose a significant threat to MOT.
- Blind-blur loss effectively achieves blind and blur attacks, and the attacked tracker faces the dilemma of completely missing the target and focusing on the background.
- Our method has excellent transferability. Experiments on the benchmark datasets show that our attack method can reduce the performance of TBD-based and JDT-based trackers, even though it is not designed for them.

2. Related works

2.1. Multi-object tracking

Tracking-By-Detection. This paradigm is indispensable for the development of MOT, and most trackers (Bewley et al., 2016; Porzi, Hofinger, Ruiz, Serrat, Bulò, & Kotschieder, 2019; Ristani & Tomasi, 2018; Shuai et al., 2021; Wojke et al., 2017; Xu et al., 2019; Zhang et al., 2021; Zhu et al., 2019) have followed it in the past few years. They address the tracking problem in two steps: first, use an excellent detector like FasterRCNN (Ren et al., 2017), SSD (Liu et al., 2016), and YOLO (Ge et al., 2021; Redmon & Farhadi, 2018; YOLOv5, 2020) to search for the target in every frame. And then, the detection boxes of adjacent frames are made in association with the ReID system. SORT (Bewley et al., 2016) is a typical representative of this paradigm, mainly focusing on Kalman filters and Hungarian algorithms to predict the position of targets. After that, more and more trackers concentrate on object association component. Ristani and Tomasi (2018) adopt a convolutional neural network to learn useful features for ReID. Xu et al. (2019) propose a spatial-temporal relation network to build robust similarity scores. Zhu et al. (2019) propose a unified framework integrating single object tracking and data association method. SiamMOT (Shuai et al., 2021) is a new tracker based on SORT, which constructs a motion model to predict the trajectory. These trackers achieve the top performance with the help of high-performance detectors and robust association modules. ByteTrack (Zhang et al., 2021), for example, combines the leading performance YOLOX with the efficient ReID system to achieve the latest top on most benchmarks.

Joint-Detection-Tracking. The popular trackers (Guo, Wang, Wang, & Tao, 2021; Lu et al., 2020; Mukhtar & Khan, 2023; Peng, Wang, Wan, Wu, & Fu, 2020; Wang, Zheng, et al., 2020; Wang, Zheng, Pan, & Xu, 2021; Wu et al., 2021; Zhang et al., 2018, 2020; Zhou et al., 2020) adopt this paradigm for its resourceful and unified framework. Most online JDT-based trackers bind excellent detectors with additional modules for data fusion, promoting the network to perform two tasks simultaneously. Zhang et al. (2018) use the tracked bounding boxes to improve detection and adopt bipartite-matching for the bounding-box association. JDE (Wang, Zheng, et al., 2020), CTracker (Peng et al., 2020), RetinaTrack (Lu et al., 2020), CenterTrack (Zhou et al., 2020), FairMOT (Zhang et al., 2020), CorrTracker (Wang, Zheng, Pan, & Xu, 2021) and Guo, Wang, Wang, and Tao (2021) introduce a ReID module or ID verification branch into a single-shot detector to achieve high accuracy and fast running speed.

TraDeS (Wu et al., 2021) adopts CenterNet (Zhou, Wang, & Krhenbühl, 2019) as its object detector to represent each point on the feature map with the object center or a background region. To build an end-to-end tracker, TraDeS proposes Cost Volume Based Association (CVA) module and Motion-guided Feature Warper (MFW) module for learning ReID embedding and tracking trajectories, respectively. TraDeS achieves great performance on most tracking benchmark datasets, thereby providing a new strategy in the tracking paradigm family.

2.2. Adversarial attack

The first adversarial attack method for deep learning models, as explored in the field of image classification by Szegedy et al. (2013). Following that, several works (Goodfellow, Shlens, & Szegedy, 2014; Kurakin et al., 2016; Moosavi-Dezfooli et al., 2016; Wei et al., 2018) emerged, primarily focusing on optimizing an antagonistic objective function. These methods require multiple iterations to generate adversarial examples, the typical work is FGSM (Goodfellow, Shlens, & Szegedy, 2014). Recently, the method (Guo, Cheng, et al., 2021; He et al., 2023; Liang et al., 2020; Wei et al., 2018; Xiao et al., 2018; Yan et al., 2020) based on the generation model has attracted more and more attention due to the rapid generation of adversarial examples. AdvGAN (Xiao et al., 2018) is the first attack method to train a noise generator with GAN (Goodfellow, Pouget-Abadie, et al., 2014), which generates high-quality adversarial examples quickly. Since then, adversarial attacks have gradually been applied to other visual tasks. CSA (Yan et al., 2020) proposes a noise generator with U-Net (Ronneberger, Fischer, & Brox, 2015) to produce adversarial examples, which successfully deceive the state-of-the-art models in single object tracking. UEA (Wei et al., 2018) proposes a generative method to produce adversarial images and videos for object detection. Wang, Wang, Li, Zhang, and Lin (2020) implement an efficient attack method on the ReID system based on GAN.

In the field of MOT, adversarial attacks focus on against object detector and association component directly. Jia et al. (2019) propose an attack method to deceive Kalman filter by moving the original detection boxes towards the direction of decreased tracking performance. Although this method has excellent performance in combating the continuity of object identification, its effectiveness is limited when attacking the tracker without a Kalman filter. Recently, Zhou et al. (2023) propose another attack method called F&F, breaking this limitation by fooling detectors alone and treating the associated component as a black-box. F&F erases the original detections and injects deceptive false boxes, making a greater number of ID switches. Unlike previous methods that target ground attack detectors or associated components, our method achieves more effective attacks by disrupting the motion prediction of trackers. And our attack effect is not limited to ID switching, but rather blinding and blurring trackers.

3. Proposed method

In this section, we first review MOT and the attacked tracker TraDeS (Wu et al., 2021). Then, we explain the proposed attack mechanism of blinding and blurring tracker. Specifically, we propose a blind-blur loss combined with tracking information for end-to-end attacking. Going a step further, we additionally design an adversarial detection loss for attacking the detector in MOT. The adversarial examples trained for attacking detector are then used to attack the multi-object trackers. The implementation details of our attack method are shown in Fig. 1.

3.1. Overview of multi-object tracker

Multi-object trackers are used to predicting the trajectories of targets in a video sequence, which can be formally expressed as $\{T_1, T_2, \dots, T_N\}$. T_i represents the entire trajectory of an object (ith), it consists of multiple boxes $\{B_i^1, B_i^2, \dots, B_i^T\}$ and a fixed ID. TBD-based trackers realize that the detection and tracking are two independent networks, but JDT-based trackers integrate them in a unified network. To be specific, in TBD, the image $\{I^t\}_{t=1}^T \in \mathbb{R}^{W \times H \times 3}$ are first input to detection network for getting a set of detected instances $\{B_i^t\}_{i=1:N}^{t=1:T}$. Then these candidate boxes from frame F^t and frame F^{t+1} are collected and converted to motion trajectories by similarity measurement algorithm and data fusion algorithm like Kalman Filter. Unlike TBD, JDT-based trackers produce detection boxes, ReID embeddings, and

tracking offsets simultaneously. For example, CenterTrack (Zhou et al., 2020) introduces a tracking offset branch to CenterNet (Zhou et al., 2020) and outputs tracking offsets $\{O_i^t\}_{i=1:N}^{t=1:T}$ and detection boxes $\{B_i^t\}_{i=1:N}^{t=1:T}$.

The official evaluation metrics of MOTChallenge are Multiple Object Tracking Accuracy (MOTA) and IDF1 Score (IDF1). MOTA combines three error sources: False Negative (FN), False Positive (FP), and the total number of Identity Switches (IDSW), it can be defined as $1 - \frac{\sum_i (FN_i + FP_i + IDSW_i)}{\sum_i GT_i}$ and note that when there is enough FN, FP, and IDSW, MOTA will be marked as a negative value. $IDF1 = \frac{2IDTP}{2IDTP + IDFP + IDFN}$ represents the ID precision $\frac{TP}{TP + FP}$ and ID recall $\frac{TP}{TP + FN}$.

Here is the overall pipeline of TraDeS (Wu et al., 2021). TraDeS take images $I_i, I_{i+1} \in \mathbb{R}^{W \times H \times 3}$ from videos as the inputs and output three feature maps for tracking objects, including center heatmap $H \in \mathbb{R}^{\frac{W}{4} \times \frac{H}{4} \times 1}$, tracking offset map $O \in \mathbb{R}^{\frac{W}{4} \times \frac{H}{4} \times 2}$, and size map $S \in \mathbb{R}^{\frac{W}{4} \times \frac{H}{4} \times 2}$. Specifically, TraDeS first produce the base feature map $f \in \mathbb{R}^{\frac{W}{4} \times \frac{H}{4} \times C}$ via a backbone network $\phi(\cdot)$, and then translates feature f to ReID embedding features $E \in \mathbb{R}^{\frac{W}{4} \times \frac{H}{4} \times 128}$, center heatmap H , and size map S by a set of head convolutional branches. And tracking offset map O is produce by the CVA module and MFW module. For the base feature map f , the CVA module calculate the cost volume $C \in \mathbb{R}^{\frac{W}{4} \times \frac{H}{4} \times \frac{W}{4} \times \frac{H}{4}}$ of each point P in embedding features E . After that, the MFW module produces the tracking offset $O \in \mathbb{R}^{\frac{W}{4} \times \frac{H}{4} \times 2}$ for tracking and enhancing object features.

3.2. Blind-blur attack

In this section, the corresponding feature maps of TraDeS (Wu et al., 2021) are used to build our blind-blur loss. TraDeS takes adjacent sets of two frames as input, so that we can use this property to train adversarial examples with spatio-temporal connections. As is shown in Algorithm 1, $T^c = T_i^c, T_{i+s}^c, \dots, T_{i+ns}^c$ is the set of training data extracted from the video sequence, and s represents the distance of them in the time dimension. This means that images are not loaded according to video sequences, and using training data at different time intervals helps to enhance the adaptability of adversarial examples to different dense scenes.

The core of blind-blur loss is shown in Fig. 2. Inter-class represents different objects in frame F^t , and intra-class denotes the same object in adjacent frames (F^t and F^{t+1}). Tracking offset demonstrates how far the target has moved in the video. Based on these factors, we design three loss functions: 1. Tracking loss: shifting the tracking offset to move detections away from the original position. 2. Inter-class loss: bringing each object in frame F^t closer to its neighboring. 3. Intra-class loss: increasing detection distance between frame F^t and F^{t+1} . And they are used to build the blind loss and blur loss. The tracking offset generated by the CVA module is the change of target position from the previous frame to the current frame. In this work, we modify the offset to make the MFW module produce false movement, which leads the detector to think that the target has exceeded the field of vision. We want the predicted offset to be as far away from the actual value as possible so that the tracking loss can be written as:

$$L_{tracking}(O^c, O^a) = \max(\frac{1}{BK} \sum_{i=0}^{BK} (O_i^{cx} - O_i^{ax}), \alpha) + \max(\frac{1}{BK} \sum_{i=0}^{BK} (O_i^{cy} - O_i^{ay}), \alpha) \quad (1)$$

where $O \in \mathbb{R}^{B \times K \times 2}$ is the tracking offset of K points in $\tilde{O} \in \mathbb{R}^{B \times W \times H \times 2}$ from frame F^t to frame F^{t+n} , which is sorted according to the score predicted by heatmap. With the help of heatmap, the tracker can obtain the visibility of each object in the image. Therefore, the selected K points are actually the position markers recognized by the tracker. For

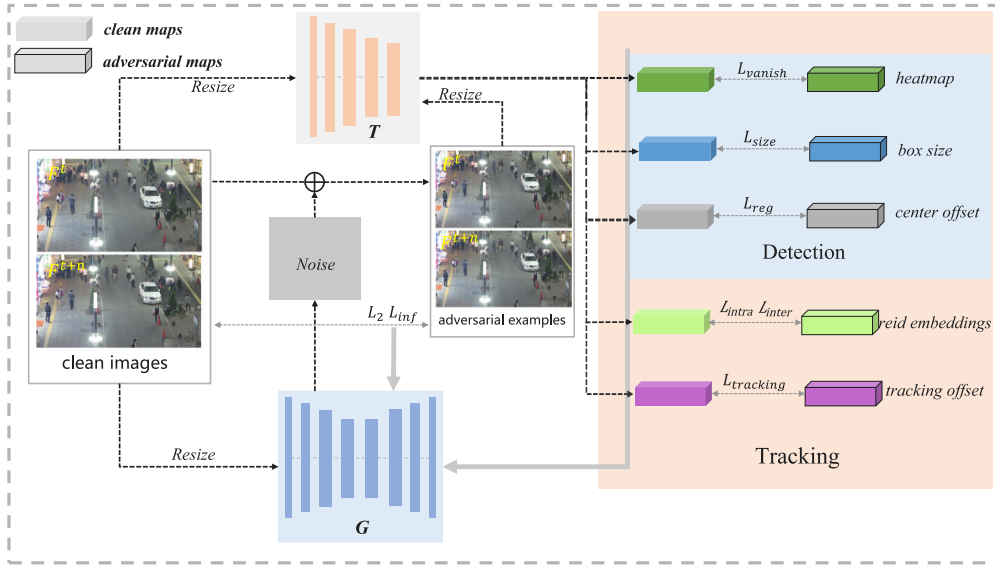


Fig. 1. Intuitive view of BBA. Clean images and adversarial examples are put into the tracker, respectively. And adversarial objective functions are constructed according to the output feature maps. Better viewed in color with zoom-in.

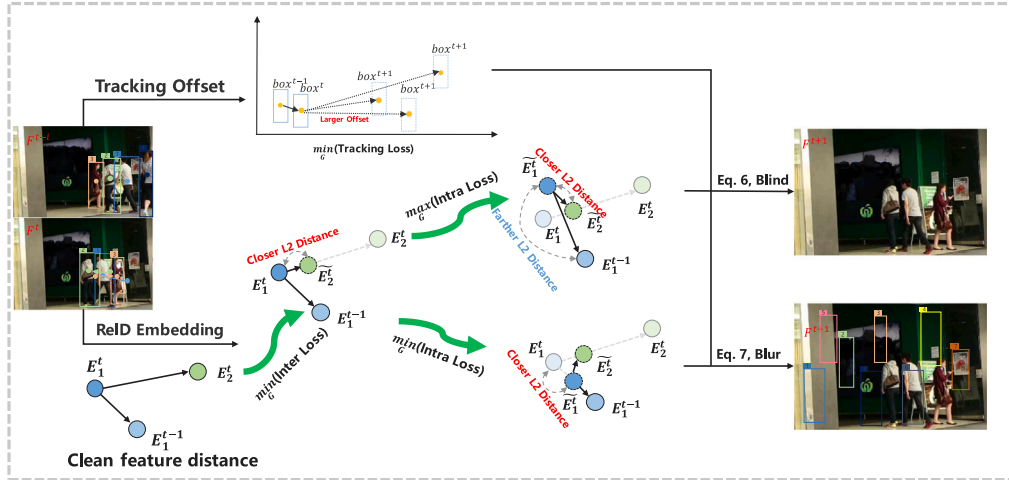


Fig. 2. Critical factors of blind-blur loss. Firstly, we amplify the predicted tracking displacement of the tracker, preventing it from accurately associating with the target. Subsequently, we minimize both intra-class and inter-class distances, effectively causing the tracker to lose sight of the target. Furthermore, by narrowing the inter-class distance and widening the intra-class distance, we render the tracker incapable of distinguishing and tracking the background, thus eroding its discriminative capabilities.

each O_i , $i \in (0, BK)$, it gracefully represents the subtle positional shifts of the target as it transitions between frames, embodying a relatively minor yet genuine numerical value. For simplicity of calculation, we express the disparities solely through the intuitive differences between these values, eschewing the introduction of any additional norms. $\alpha = -20$ is the variables for restriction. Based on the tracking loss, we design the inter-class loss and intra-class loss to achieve stronger aggression, the details are shown in Fig. 2. The objective of inter-class loss is to regulate the pairwise distances among each individual object and all remaining objects. For K detections of each image in a minibatch B , inter-class loss can be expressed as:

$$L_{inter}(E^c, E^a) = \max(\frac{1}{K} \sum_{i,j} (\|E_i^a - E_j^a\|_2 - \|E_i^c - E_j^c\|_2), \beta_1) \quad (2)$$

and for detections in each adjacent frame of a minibatch B , the intra-class loss can be written as:

$$L_{intra}(E^c, E^a) = \max(\frac{1}{B} \sum_{t=0}^{B-1} (\|E_t^a - E_{t+1}^a\|_2 - \|E_t^c - E_{t+1}^c\|_2), \beta_2) \quad (3)$$

where $E \in \mathbb{R}^{B \times K \times 128}$ is the embedding of K points in $\tilde{E} \in \mathbb{R}^{B \times W \times H \times 128}$, the selection of K is the same as that of $\tilde{O}$. $\beta_1 = -2.5$ and $\beta_2 = -3.5$ are the boundaries of similarity changing. The embedding $E_{i=0:K} \in \mathbb{R}^{B \times 1 \times 128}$ in $E \in \mathbb{R}^{B \times K \times 128}$ is the feature of K different objects in frame F^t . The embedding $E_{i=0:B} \in \mathbb{R}^{K \times 1 \times 128}$ in $E \in \mathbb{R}^{B \times K \times 128}$ represent the feature of objects in adjacent frames F^t and F^{t+1} . The added noise may change the confidence ranking of targets in E , so it is possible to increase or reduce the intra-class loss. To test our conjecture, we construct two objective functions: $F_{blind} = \min(L_{tracking}) + \min(L_{inter}) + \min(L_{intra})$ and $F_{blur} = \min(L_{tracking}) + \min(L_{inter}) + \max(L_{intra})$. In addition, we design the L_{inf} loss and improved L_2 loss to limit noise. The L_{inf} and L_2 can be expressed as:

$$L_{inf}(I^c, I^a) = \max(|I_i^c - I_i^a|) \quad (4)$$

$$L_2(I^c, I^a) = \frac{1}{n} \sum_{i=1}^n (I_i^c - I_i^a)^2 \exp(I_i^c) \quad (5)$$

where I^c and I^a represent normalized clean image and adversarial example, respectively. $\exp(I_i^c)$ is used to limit pixels with significant

Algorithm 1 Framework of training perturbation generator with video sequences

Input: $\{I^c\}$: Clean image set from video sequences; G_0 : Randomly initialized generator;

Output: G : Trained generator;

```

1: Initialize generator  $G_0$  to  $G$ , load model for tracker  $T$ ;
2: while  $G$  is not converge do
3:   for  $\{I^c\}$  in training dataset do
4:     Feed  $\{I^c\}$  into  $T$ ;
5:     Get clean embedding feature maps  $E^c$  and tracking offset maps  $O^c$ ;
6:     Feed  $\{I^c\}$  into  $G$ ;
7:     Get adversarial examples  $\{I^a\}$ ;
8:     Feed  $\{I^a\}$  into  $T$ ;
9:     Get attacked embedding feature maps  $E^a$  and tracking offset maps  $O^a$ ;
10:    Calculate adversarial noise loss  $L_2$ ,  $L_{inf}$  and blind-blur loss  $L_{blind-blur}$  based on  $E^c$ ,  $O^c$ ,  $E^a$ , and  $O^a$ ;
11:    Calculate gradient of loss to generator parameters of  $G$  and update with Adaptive Moment Estimation(Adam) optimizer.
12:  end for
13: end while
14: return  $G$ 

```

changes and avoid excessive concentration of noise in certain areas. n is the number of pixels in the image. To sum up, the blind-blur loss can be written as:

$$L_{blind} = \lambda_1 L_2 + \lambda_2 L_{inf} + \lambda_3 L_{inter} + \lambda_4 L_{intra} + \lambda_5 L_{tracking} \quad (6)$$

$$L_{blur} = \omega_1 L_2 + \omega_2 L_{inf} + \omega_3 L_{inter} + \omega_4 L_{intra} + \omega_5 L_{tracking} \quad (7)$$

where λ_i and ω_i , $i \in (1, 5)$ are the weights of blind-blur loss, change their combinations can achieve different noise scales and attack visualization style. Specifically, the blind-blur loss alters the object relationships across frames and spaces by controlling the optimization direction of intra-class and inter-class losses, which effectively constrains the tracker's capability to retain object localization and identity recognition. The blind-blur loss consists of opposite blind and blur losses, which are independently performed during the training of the noise generator. When conducting blind attacks on the tracker, the main objective of optimization is blind loss, while the blur loss is used for blurring.

3.3. Object detection attack

CenterNet (Zhou et al., 2019) is a popular anchor-free object detector in MOT. It estimates the position of each object through the center point (x, y) , offset o , and size (w, h) predicted by multiple heads, they can be denoted as $bbox_i = (x_i + o_i, y_i + o_i, w_i, h_i)$. For the object detection problem, we design an adversarial detection loss composed of the vanishing loss and shrinkage loss to cool the center-heatmap and shrink the detection box. The vanish loss aims to move or disappear points in the heatmap, and it consists of three parts:

$$\begin{aligned}
L_{vanish}(S^a, S^c, C^a, C^c, I^a, I^c) = & \max(\epsilon_1 \frac{1}{K} \sum_{i=1}^K (S_i^a - S_i^c), \epsilon_2) \\
& + \frac{1}{K} \sum_{i=1}^K ModSign(C_i^a - C_i^c) \\
& + \frac{1}{K} \frac{\xi_1}{\sqrt{\sum_{i=1}^K (I_i^c - I_i^a)^2 + \xi_2}}
\end{aligned} \quad (8)$$

where $\epsilon_1 = 0.01$, $\epsilon_2 = -10$, $\xi_1 = 50$, and $\xi_2 = 5e - 4$ control the maximum change in the centrality heatmap. For each heatmap

$H \in \mathbb{R}^{\frac{W}{4} \times \frac{H}{4} \times C}$, where W and H represent the width and height of the original image, respectively, and C is the number of distinct object classes, we select the top K values with the highest confidences as potential detections. These detections are decoded into three primary maps: the score map $S \in \mathbb{R}^{K \times 1}$ indicating detection probabilities, the classification map $C \in \mathbb{R}^{K \times 1}$ identifying predicted object labels, and the index map $I \in \mathbb{R}^{K \times 1}$ pinpointing target positions. We devise a strategy to undermine the heatmap attack detector. Firstly, we reduce values in the score map S to weaken confidence, causing missed detections. Secondly, we manipulate classification info in the classification map C , leading to misclassification. Lastly, we adjust positions on the index map I to deviate from their original values, compromising localization. To capture the intricate variations of C , we introduce a modified sign function $ModSign(\cdot)$, meticulously defined as follows:

$$ModSign(x) = \begin{cases} 1, & \text{if } x = 0 \\ 0, & \text{otherwise} \end{cases} \quad (9)$$

where x summarizes the differences in classification before and after the attack, L_{vanish} only focuses on whether it is zero and is not sensitive to other specific values. Based on the vanish loss, an offset loss is proposed to improve the attack capability against samples. We write the offset loss as:

$$\begin{aligned}
L_{reg}(O^a, O^c) = & \max(\frac{\eta_1}{WH} \sum_{i=1}^{WH} (O_i^{cx} - O_i^{ax}), \eta_2) \\
& + \max(\frac{\eta_1}{WH} \sum_{j=1}^{WH} (O_j^{cy} - O_j^{ay}), \eta_2)
\end{aligned} \quad (10)$$

where O^a and $O^c \in \mathbb{R}^{1 \times 2 \times W \times H}$ are the output of box offset head before and after attack. O_i^{cx} and $O_j^{cy} \in \mathbb{R}^{1 \times WH}$ are the horizontal and vertical displacements predicted by offset O^c , respectively. $\eta_1 = 100$ and $\eta_2 = -25$ control the offset component. Similar to $L_{tracking}$, the goal of L_{reg} is also to increase the offset, but here it represents the gap mapped to the original image after upsampling the position on the heatmap. Then we also propose a size loss to shrink the prediction box so that the tracking cannot accurately cover the targets. We use the output of size head after being attacked to build loss, which can be written as:

$$L_{size}(S^a, S^c) = \max(\frac{1}{WH} (\zeta_1 (S_i^a - S_i^c)), \zeta_2), S_i = 0 \text{ if } S_i < \zeta_3. \quad (11)$$

where $S^a \in \mathbb{R}^{1 \times 2 \times W \times H}$ denotes the size map after attack. $\zeta_1 = 0.1$ and $\zeta_2 = -40$ control the change of size. ζ_3 is the target size threshold, which set to 10 and want to attack primarily larger targets. Finally, the adversarial detection loss can be expressed as:

$$L_{adv_detect} = \lambda_1 L_2 + \lambda_2 L_{inf} + \lambda_3 L_{vanish} + \lambda_4 L_{reg} + \lambda_5 L_{size} \quad (12)$$

where λ_i , $i \in (1, 5)$ are the weights of different losses in adversarial detection loss. Changing the weight combinations can achieve different levels of noise scales and attack performance.

4. Experiments

We implement BBA on TraDeS (Wu et al., 2021) and transferred it to CenterTrack (Zhou et al., 2020), FairMOT (Zhang et al., 2020), and ByteTrack (Zhang et al., 2021) as black-box attacks. Three challenging datasets are used to validate the performance of our method, including MOT16 (Milan, Leal-Taixe, Reid, Roth, & Schindler, 2016), MOT17 (Milan et al., 2016), and MOT20 (Dendorfer et al., 2020). Besides, we also train a generator to attack object detection CenterNet (Zhou et al., 2019). Detailed results are provided as follows.

4.1. Experiment setup

Training generator. We choose the U-Net (Ronneberger et al., 2015) as our generator inspired by CSA (Yan et al., 2020). To effectively attack TraDeS (Wu et al., 2021), we combine images with different

Table 1

Comparison of tracker performance under different attacks.

Metrics	Original	Attack det	Blind	Blur
MOTA↑	68.2	32.0	4.7	-399.2
IDF1↑	71.7	39.5	7.0	4.3
IDs↓	285	278	315	1393

frame spacing from MOT17 (Milan et al., 2016) to learn spatial-temporal information. Specifically, due to memory limitations (11 GB), we set the batch size to 6, i.e., three image pairs with 2, 1, and -1 frame intervals. The images of MOT17 are large, and we reshape them to 512×256 before being put into the generator. When we get the output noise, we reshape it and add it to the original image. Experiments show that this method can improve the generator's efficiency without losing precision. For the ablation study, we also train a generator with CrowdHuman (Shao et al., 2018) and MOT17 (Milan et al., 2016) datasets to attack CenterNet (Zhou et al., 2019), the details remaining the same.

Implementation Details. The experiments are implemented in Python with Pytorch on 2 RTX2080ti. We use the adaptive moment estimation (Adam) to train our generator for 50 epochs with an initial learning rate of 0.0002. For λ_i , $i \in (1, 5)$ in blind loss, we set 1000, 20, 8, 12, and 1.2, respectively. And for the blur loss, we set the weights ω_i , $i \in (1, 5)$ to 500, 20, 40, -40, and 1.5, respectively. In adversarial detection loss, we set $\lambda_1 = 500$, $\lambda_2 = 10$, $\lambda_3 = 1$, $\lambda_4 = 1.5$, and $\lambda_5 = 4$.

Datasets and Metrics. In this work, we adopt five datasets for training and verification, they are CrowdHuman (Shao et al., 2018), COCO (Lin, Maire, Belongie, Hays, & Zitnick, 2014), MOT16 (Milan et al., 2016), MOT17 (Milan et al., 2016), and MOT20 (Dendorfer et al., 2020). Specifically, CrowdHuman and COCO are used to train the generator to attack CenterNet (Zhou et al., 2019) and the MOT datasets are used to train and test BBA. For the ablation study, we divide the MOT17 dataset into training subsets and validation subsets inspired by TraDeS (Wu et al., 2021). The metrics of MOT are MOTA, IDF1, HOTA, MT, ML, and IDS. The main evaluation metrics of COCO are average precision (AP) and mean average precision (mAP). To improve the visibility of attack effectiveness, we use the difference between clean and attacked metrics to represent it, which can be simply expressed as $\Delta M = M_{clean} - M_{attacked}$. Going further, the rate of change can be described as $\Delta M\% = \frac{M_{clean} - M_{attacked}}{M_{clean}}$.

4.2. Ablation study

Effectiveness Analysis of blind-blur Loss. As introduced in Section 3.2, the blind-blur loss makes the tracker lose targets and locate background by switching the direction parameters. As is shown in Figs. 3(b) and 3(c), *blind* and *blur* display different attack styles when attacking TraDeS (Wu et al., 2021). In comparison to the visualization of the original tracking results, the blinded tracker misclassifies pedestrians in the image as background, leading to a significant reduction in the number of tracking boxes. Instead, a blurred tracker erroneously identifies the background region as targets of interest, utilizing multitude tracking boxes to label them. Regarding attacks on the detector, they merely cause the tracker to lose a small portion of the pedestrian tracking boxes, while the tracking capabilities are still partially preserved. We compare the blind-blur loss with the adversarial detection loss under the same disturbance level, the detailed results are shown in Table 1. The results with a large gap fully illustrate the effectiveness of the blind-blur loss.

Effectiveness Analysis of Detection Loss. As shown in Fig. 3(d), attacks against the TraDeS detector lets it lose most of its targets. The MOTA obtains a 53.23% drop, indicating that the performance of the detector is closely related to the tracker. To fully explore the performance of the detection loss, we train an additional noise generator to attack object detection CenterNet (Zhou et al., 2019). On the COCO (Lin

Table 2

Comparison of AP before and after detection attack of CenterNet (Zhou et al., 2019), YOLOv5-x (YOLOv5, 2020), and YOLOX-X (Ge et al., 2021) on COCO validation set. ‘AD’ represents attack detector. MAE represents mean absolute error.

Detector	Method	AP↓	AP ₅₀ ↓	AP ₇₅ ↓	MAE↓	Type
CenterNet (Zhou et al., 2019)	Clean	36.6	54.3	39.2	—	—
	UEA	28.6	43.4	30.1	8.45	Black-box
	Ours	14.8	23.7	15.5	7.13	White-box
YOLOv5x (YOLOv5, 2020)	Clean	50.7	68.9	55.0	—	—
	UEA	40.9	57.9	44.1	8.51	Black-box
	Ours	38.6	55.6	41.4	7.22	Black-box
YOLOX-X (Ge et al., 2021)	Clean	51.1	69.4	55.3	—	—
	UEA	44.5	61.3	48.1	8.38	Black-box
	Ours	42.1	59.7	45.2	7.19	Black-box

et al., 2014) verification set, the AP of CenterNet after the attack is only 14.8, dropped 59.56%. In addition, we transplant the detection attack method to other detectors, including YOLOv5 (YOLOv5, 2020) and YOLOX (Ge et al., 2021). And we compare our method with UEA (Wei et al., 2018). The detailed results are shown in Table 2.

Noise levels. To explore the relationship between noise visibility and attack effect, we visualize three noise levels. We change the level of added noise by controlling the weight of L_2 loss in blind-blur loss. Specifically, during the training phase, the weights of L_2 in the blur loss are set to 1500, 1000, and 500, respectively. The mean absolute errors (MAE) corresponding to the three scale noises are 1.87, 3.86 and 5.58, respectively. As is shown in Fig. 4, they are light, moderate, and heavy noise. We show the noise performance on the MOT17 validation set for the *blur* attack. Using the light noise in Fig. 4(b) to attack TraDeS (Wu et al., 2021), the MOTA down to -105.9 and IDF1 descend to 31.6. As shown in Fig. 4(c), a moderate level noise makes MOTA drop to -376.4 and IDF1 down to 7.8, respectively. The heavy noise in Fig. 4(d) is used to explore the limiting performance of our disturbance. MOTA of the attacked tracker is down to -399.2, and IDF is down to 4.3. The heavy noise performed only slightly better than the moderate, indicating that our noise is efficient.

Performance on different paradigms. To test the transferability of BBA, we apply our attack method to TBD-based and JTD-based trackers. CenterTrack (Zhou et al., 2020) and FairMOT (Zhang et al., 2020) are being to JTD. ByteTrack (Zhang et al., 2021) is a state-of-the-art TBD-based tracker. TraDeS is based on CenterTrack, and our attack methods remain effective against CenterTrack. FairMOT introduces an additional ReID branch for reidentification and BBA can drop its performance by about 50%. ByteTrack correlated all the boxes regardless of confidence, which resulted in the ineffectiveness of our attack, and make the performance of *blind* to degrade. However, the *blur* attack can still make MOTA to plummet from 76.6 to -1253.6, mainly due to a large number of wrong detection boxes that interfered with fusion. The detailed results can be shown in Table 3.

4.3. MOT challenge results

We show the attack performance of BBA on the test set of MOTChallenge benchmarks, including MOT16 (Milan et al., 2016), MOT17 (Milan et al., 2016), and MOT20 (Dendorfer et al., 2020). To our best knowledge, we are the first to attack both TBD-based and JTD-based trackers on the MOTChallenge benchmark datasets.

MOT16. The MOT16 test set consists of 7 video sequences, and we evaluate our attack methods on it against TraDeS (Wu et al., 2021). As is shown in Table 4, the MOTA of TraDeS gets a 67.1 and 316.2 drop, respectively. At the same time, the IDF1 has a 60.4, and 60.6 decrease. The *blind* makes the IDs from 1144 to 225, which main reason is the significant reduction in detection boxes caused by blinding attacks.

MOT17. On the MOT17 dataset, we show the attacks for TraDeS (Wu et al., 2021), FairMOT (Zhang et al., 2020), and ByteTrack (Zhang et al., 2021). MOT17 dataset contains video sequences from MOT16,



Fig. 3. Visualization of attacks. Attack det means attack CenterNet of Tracker.

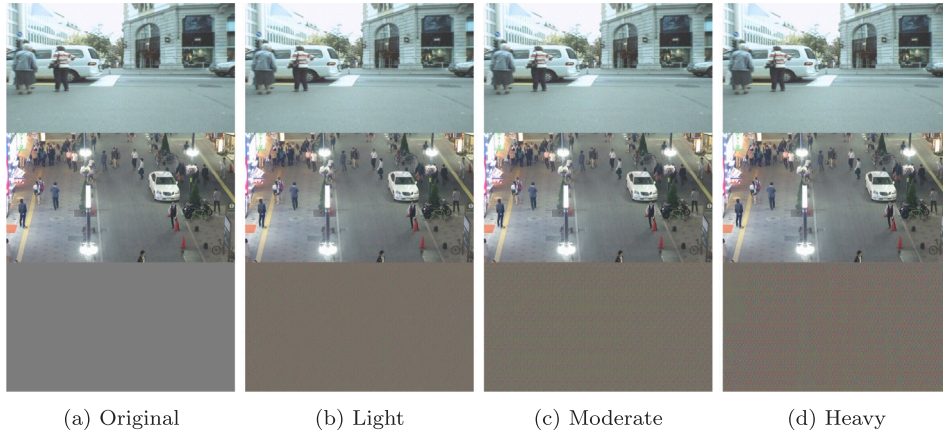


Fig. 4. Three different levels of noise.

Table 3

Comparison the MOTA, IDF1, and IDs of TraDeS (Wu et al., 2021), FairMOT (Zhang et al., 2020), and ByteTrack (Zhang et al., 2021) before and after attacks on MOT17 (Milan et al., 2016) validation set.

Method	MOTA↑			IDF1↑			IDs↓		
	Clean	Blind	Blur	Clean	Blind	Blur	Clean	Blind	Blur
TraDeS (Wu et al., 2021)	68.2	4.7	−399.2	71.7	7.0	4.3	285	315	1393
CenterTrack (Zhou et al., 2020)	66.1	8.7	−236.7	64.2	10.4	7.4	528	436	920
FairMOT (Zhang et al., 2020)	69.1	31.3	31.0	72.8	35.0	33.4	299	470	1130
ByteTrack (Zhang et al., 2021)	76.6	73.6	−1252.6	79.3	73.3	7.3	159	198	543

and our attack can still effectively trick TraDeS. At most, our attack method can achieve a maximum of 444.57%, 73.13%, and 544.59% change of Δ MOTA of TraDeS, FairMOT, and ByteTrack. At the same time, it also poses a threat to Δ IDF1, reaching 93.42%, 72.0%, and 77.75%, respectively. The sharply increased IDs fully demonstrates the effectiveness of BBA in disrupting the tracker. The specific results are shown in Table 4.

MOT20. MOT20 has more complex video scenes than MOT17. We attack and evaluate FairMOT (Zhang et al., 2020) and ByteTrack (Zhang et al., 2021) on it. As is shown in Table 4, our attack method further degrades the performance of trackers, which means that our noise can perform better in complex scenarios. For example, MOTA of blur-attacked FairMOT decreased from 61.8 to 7.2, 15.22% more degradation than MOT17.

4.4. Further discussions

Speed. Our method also has the advantage of speed. The generator produces an adversarial example $I^a \in \mathbb{R}^{3 \times W \times H}$, which takes less than

3 ms. Our attack method is faster than most mainstream trackers and does not cause a significant drop in FPS. So a real-time attack is not easy to detect.

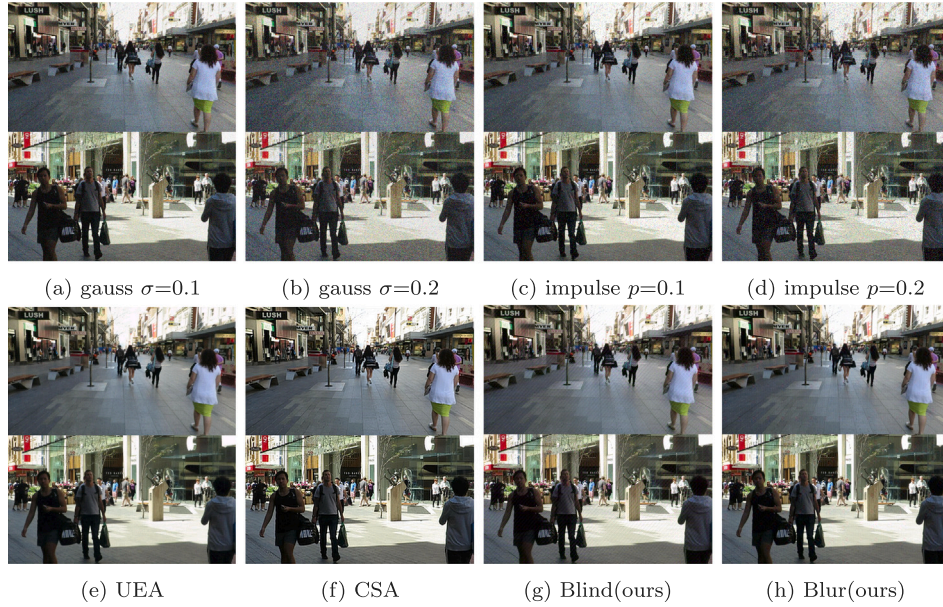
Frames of attack. For video sequences, we implement attack every {2,4,8,16, 32} frames to make the attack subtle. As shown in Table 5, we evaluate the performance of *blind* and *blur*. *Blur* performs better than *blind* in a differential frame attack, and IDs increase significantly when the attack is carried out one frame apart.

Comparison with Other Methods. We compare our method with effective methods for object detection and object tracking. At the same time, heavy random noise is also used to compare, including impulse noise and gaussian noise. UEA proposes a method for effectively attacking object detection in images and videos. CSA (Yan et al., 2020) is a very effective method in single object tracking attack method. As shown in Fig. 5, the noises we generate are less obvious than others but more destructive. Table 6 shows the results of different noises against TraDeS (Wu et al., 2021). Compared with other noises under the MOT evaluation metric, our method can significantly degrade the performance of trackers.

Table 4

Compare TraDeS (Wu et al., 2021), CenterTrack (Zhou et al., 2020), FairMOT (Zhang et al., 2020), and ByteTrack (Zhang et al., 2021) before and after attack under the “private detector” protocol on MOT test set.

Dataset	Method	MOTA↑	IDF1↑	HOTA↑	MT↑	ML↓	IDs↓
MOT16 (Milan et al., 2016)	TraDeS (Wu et al., 2021)	70.1	64.7	53.2	37.3%	20.0%	1144
	TraDeS + <i>blind</i>	3.0	4.3	6.4	0.1%	95.8%	225
	TraDeS + <i>blur</i>	−246.1	4.1	6.1	0.1%	76.5%	1895
MOT17 (Milan et al., 2016)	TraDeS (Wu et al., 2021)	69.1	63.9	52.7	36.4%	21.5%	3555
	TraDeS + <i>blind</i>	3.0	4.2	6.4	0.1%	95.9%	681
	TraDeS + <i>blur</i>	−238.1	4.4	6.2	0%	77.7%	5831
	FairMOT (Zhang et al., 2020)	73.7	72.3	59.3	43.2%	17.3%	3303
	FairMOT + <i>blind</i>	31.4	33.3	27.2	5.0%	62.9%	3057
	FairMOT + <i>blur</i>	19.8	23.3	19.5	1.4%	69.0%	3120
	ByteTrack (Zhang et al., 2021)	80.3	77.3	63.1	53.2%	14.5%	2196
	ByteTrack + <i>blind</i>	75.6	72.8	58.3	41.5%	19.9%	2205
	ByteTrack + <i>blur</i>	−357.0	17.2	20.9	26.7%	36.7%	4831
MOT20 (Dendorfer et al., 2020)	FairMOT (Zhang et al., 2020)	61.8	67.3	54.6	68.8%	7.6%	5243
	FairMOT + <i>blind</i>	7.9	9.9	10.9	0.6%	6.1%	1288
	FairMOT + <i>blur</i>	7.2	8.8	10.4	0.8%	87.8%	1236
	ByteTrack (Zhang et al., 2021)	77.8	75.2	61.3	69.2%	9.5%	1223
	ByteTrack + <i>blind</i>	72.5	69.9	56.5	59.8%	12.9%	1923
	ByteTrack + <i>blur</i>	2.6	40.0	35.4	51.4%	13.9%	4910

**Fig. 5.** Compare our adversarial noise with other kinds of noises.**Table 5**

Tracking results under attacking the different number of frames. Atts.Num represents the probability of being attacked in each frame.

	Att.Num	0	1	$\frac{1}{2}$	$\frac{1}{4}$	$\frac{1}{8}$	$\frac{1}{16}$	$\frac{1}{32}$
Blind	MOTA↑	68.2	4.7	35.0	51.5	59.5	63.8	66.4
	IDF1↑	71.7	7.0	42.7	55.7	59.5	64.4	66.9
	IDs↓	285	315	1295	857	606	461	377
Blur	MOTA↑	68.2	−399.2	−160.8	−27.3	20.0	44.0	56.6
	IDF1↑	71.7	4.3	15.0	32.4	43.1	49.5	57.2
	IDs↓	285	1393	5738	3416	1865	1081	657

We undertake a comparative analysis of our proposed BBA attack with F&F (Zhou et al., 2023) and Hijacking (Jia et al., 2019) within the evaluation framework established by F&F. The comparative results are presented in Table 7. When conducting attacks by targeting 3 frames every 30 frames, our black box attack method significantly outperforms the white box Hijacking and F&F attacks in terms of MOTA degradation. Notably, our blur attacks effectively divert the focus of the object tracking system towards the background, resulting in a substantial increase in false positives (FP). Furthermore, unlike

Table 6

Comparison with other methods. MAE represents mean absolute error.

Method	Δ MOTA↑	Δ IDF1↑	Δ IDs↑	MAE↓
Gauss sigma=0.1	36.7	32.9	98	19.34
Gauss sigma=0.2	61.5	62.1	−127	38.05
Impulse p=0.1	37.1	32.5	123	11.89
Impulse p=0.2	57.9	59.3	−97	22.25
UEA (Wei et al., 2018)	17.7	27.2	33	8.45
CSA (Yan et al., 2020)	9.2	8.4	54	5.63
Blind(ours)	63.5	64.7	30	4.12
Blur(ours)	467.4	67.4	1108	5.58

Table 7

Comparison with Hijacking (Jia et al., 2019) and F&F (Zhou et al., 2023) on MOT17(Milan et al., 2016).

Method	Δ MOTA↑	Δ IDF1↑	Δ IDSW(%)↑	Δ FN(%)↑	Δ FP(%)↑
Hijacking	3.12	10.30	1.96	1.1	0.66
F&F	3.59	13.82	2.95	−0.53	1.17
Blur(ours)	7.72	5.27	0.62	1.28	6.32

F&F, which necessitates prior knowledge of detector outputs, our attack method can be executed without any prior knowledge of the target tracker, thereby posing a more significant threat. Although F&F excels in switching pedestrian identities and reducing IDF1 scores, effectively disabling the identity recognition capabilities of the tracking system, it still retains its detection functionality.

5. Conclusion

In this paper, we present an efficient and effective adversarial attack method for tricking multi-object trackers. We thoroughly analyzed TraDeS and proposed blind-blur loss for two types of attacks. By optimizing the control of inter-class loss and intra-class loss, the attacked tracker was able to switch between object loss and background obsession. Our attack method is applicable to TraDeS and has strong transferability, which can be used as a black box attack method to transfer to other trackers such as CenterTrack, FairMOT, and ByteTrack. In addition, our method also poses a threat to commonly used detectors in MOT, such as CenterNet and YOLOX. This is crucial for the study of the robustness of trackers, and we believe that this work will contribute to the development of multi-object tracking and adversarial attacks.

CRedit authorship contribution statement

Haibo Pang: Methodology, Project administration. **Rongqi Ma:** Conceptualization, Methodology, Resources, Software, Writing – original draft, Writing – review & editing. **Jie Su:** Resources, Supervision, Visualization, Writing – original draft. **Chengming Liu:** Resources. **Yufei Gao:** Writing – review & editing. **Qun Jin:** Investigation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This work was supported in part by the National Key Research and Development Program of China in 2020 under Grant No. 2020YFB1712401, Zhengzhou University Offline Excellent Course Project in 2019 under Grant No. 2019ZZUXK023, Zhengzhou University Curriculum Ideological and Political Project in 2021 under Grant No. 2021ZZUGS KCSZ037, and Postgraduate Education Reform and Quality Improvement Project of Henan Province in 2022 under Grant No. YJS2022ZX04, and China Scholarship Council.

References

Bewley, A., Ge, Z., Ott, L., Ramos, F., & Upcroft, B. (2016). Simple online and realtime tracking. In *2016 IEEE international conference on image processing*.

Chen, L. C., Papandreou, G., Kokkinos, I., Murphy, K., & Yuille, A. L. (2018). DeepLab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected CRFs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(4), 834–848.

Dendorfer, P., Rezatofighi, H., Milan, A., Shi, J., Cremers, D., Reid, I., et al. (2020). MOT20: A benchmark for multi object tracking in crowded scenes. *arXiv*.

Duan, R., Mao, X., Qin, A. K., Yang, Y., Chen, Y., Ye, S., et al. (2021). Adversarial laser beam: Effective physical-world attack to DNNs in a blink.

Fischer, V., Kumar, M. C., Metzen, J. H., & Brox, T. (2017). Adversarial examples for semantic image segmentation.

Ge, Z., Liu, S., Wang, F., Li, Z., & Sun, J. (2021). YOLOX: Exceeding YOLO series in 2021. *arXiv preprint arXiv:2107.08430*.

Goodfellow, I. J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., et al. (2014). Generative adversarial networks. *Advances in Neural Information Processing Systems*, 3, 2672–2680.

Goodfellow, I. J., Shlens, J., & Szegedy, C. (2014). Explaining and harnessing adversarial examples. *arXiv preprint arXiv:1412.6572* 2014.

Guo, Q., Cheng, Z., Juefei-Xu, F., Ma, L., & Zhao, J. (2021). Learning to adversarially blur visual object tracking.

Guo, S., Wang, J., Wang, X., & Tao, D. (2021). Online multiple object tracking with cross-task synergy.

He, L., Ai, Q., Yang, X., Ren, Y., Wang, Q., & Xu, Z. (2023). Boosting adversarial robustness via self-paced adversarial training. *Neural Networks*, 167, 706–714.

He, K., Zhang, X., Ren, S., & Jian, S. (2016). Identity mappings in deep residual networks. In *ECCV*.

Jia, Y., Lu, Y., Shen, J., Chen, Q. A., Zhong, Z., & Wei, T. (2019). Fooling detection alone is not enough: First adversarial attack against multiple object tracking.

Jia, S., Song, Y., Ma, C., & Yang, X. (2021). IoU attack: Towards temporally coherent black-box adversarial attack for visual object tracking. In *CVPR*.

Kurakin, A., Goodfellow, I., & Bengio, S. (2016). Adversarial examples in the physical world.

Laykaviriyakul, P., & Phaisangittisagul, E. (2023). Collaborative Defense-GAN for protecting adversarial attacks on classification system. *Expert Systems with Applications*, [ISSN: 0957-4174] 214, Article 118957. <http://dx.doi.org/10.1016/j.eswa.2022.118957>.

Liang, S., Wei, X., Yao, S., & Cao, X. (2020). Efficient adversarial attacks for visual object tracking. In *ECCV*.

Lin, T. Y., Maire, M., Belongie, S., Hays, J., Zitnick, C. L. (2014). Microsoft COCO: Common objects in context. In *European conference on computer vision*.

Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C. Y., et al. (2016). SSD: Single shot MultiBox detector. In *European conference on computer vision*.

Lu, Z., Rathod, V., Votel, R., & Huang, J. (2020). RetinaTrack: Online single stage joint detection and tracking. In *2020 IEEE/CVF conference on computer vision and pattern recognition*.

Milan, A., Leal-Taixe, L., Reid, I., Roth, S., & Schindler, K. (2016). MOT16: A benchmark for multi-object tracking.

Moosavi-Dezfooli, S. M., Fawzi, A., & Frossard, P. (2016). DeepFool: a simple and accurate method to fool deep neural networks. In *IEEE conference on computer vision and pattern recognition*.

Mukhtar, H., & Khan, M. U. G. (2023). STMMOT: Advancing multi-object tracking through spatiotemporal memory networks and multi-scale attention pyramids. *Neural Networks*, 168, 363–379.

Nakka, K. K., & Salzmann, M. (2019). Indirect local attacks for context-aware semantic segmentation networks.

Peng, J., Wang, C., Wan, F., Wu, Y., & Fu, Y. (2020). Chained-tracker: Chaining paired attentive regression results for end-to-end joint multiple-object detection and tracking.

Porzi, L., Hofinger, M., Ruiz, I., Serrat, J., Bulò, S., & Kotschieder, P. (2019). Learning multi-object tracking and segmentation from automatic annotations.

Rakai, L., Song, H., Sun, S., Zhang, W., & Yang, Y. (2022). Data association in multiple object tracking: A survey of recent techniques. *Expert Systems with Applications*, [ISSN: 0957-4174] 192, Article 116300. <http://dx.doi.org/10.1016/j.eswa.2021.116300>.

Redmon, J., & Farhadi, A. (2018). YOLOv3: An incremental improvement. *arXiv e-prints*.

Ren, S., He, K., Girshick, R., & Sun, J. (2017). Faster R-CNN: Towards real-time object detection with region proposal networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 39(6), 1137–1149.

Ristani, E., & Tomasi, C. (2018). *Features for Multi-target Multi-camera Tracking and Re-identification*. IEEE.

Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional networks for biomedical image segmentation. In *International conference on medical image computing and computer-assisted intervention*.

Shao, S., Zhao, Z., Li, B., Xiao, T., Yu, G., Zhang, X., et al. (2018). CrowdHuman: A benchmark for detecting human in a crowd.

Shuai, B., Berneshawi, A., Li, X., Modolo, D., & Tighe, J. (2021). SiamMOT: Siamese multi-object tracking. In *CVPR*.

Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. *Computer Science*.

Szegedy, C., Zaremba, W., Sutskever, I., Bruna, J., Erhan, D., Goodfellow, I., et al. (2013). Intriguing properties of neural networks. *arXiv preprint arXiv:1312.6199*.

Wang, J., Liu, A., Yin, Z., Liu, S., & Liu, X. (2021). Dual attention suppression attack: Generate adversarial camouflage in physical world.

Wang, H., Wang, G., Li, Y., Zhang, D., & Lin, L. (2020). Transferable, controllable, and inconspicuous adversarial attacks on person re-identification with deep mis-ranking.

Wang, Z., Zheng, L., Liu, Y., Li, Y., & Wang, S. (2020). Towards real-time multi-object tracking. In *Computer Vision – ECCV 2020*.

Wang, Q., Zheng, Y., Pan, P., & Xu, Y. (2021). Multiple object tracking with correlation learning.

Wei, X., Liang, S., Chen, N., & Cao, X. (2018). Transferable adversarial attacks for image and video object detection. *arXiv preprint arXiv:1811.12641*.

- Wojke, N., Bewley, A., & Paulus, D. (2017). *Simple Online and Realtime Tracking with a Deep Association Metric* (pp. 3645–3649). IEEE.
- Wu, J., Cao, J., Song, L., Wang, Y., & Yuan, J. (2021). Track to detect and segment: An online multi-object tracker.
- Xiao, C., Li, B., Zhu, J., He, W., Liu, M., & Song, D. (2018). Generating adversarial examples with adversarial networks. arXiv preprint arXiv:1801.02610.
- Xie, A. L. X. L. J. F. Y. M. A. Z. H., & Tao, D. (2019). Perceptual-sensitive GAN for generating adversarial patches. In *AAAI*.
- Xu, J., Cao, Y., Zhang, Z., & Hu, H. (2019). Spatial-temporal relation networks for multi-object tracking. In *2019 IEEE/CVF international conference on computer vision*.
- Yan, B., Wang, D., Lu, H., & Yang, X. (2020). Cooling-shrinking attack: Blinding the tracker with imperceptible noises. In *CVPR*.
- YOLOv5 (2020). <https://github.com/ultralytics/yolov5>.
- Zhang, Z., Cheng, D., Zhu, X., Lin, S., & Dai, J. (2018). Integrated object detection and tracking with tracklet-conditioned detection.
- Zhang, Y., Sun, P., Jiang, Y., Yu, D., Yuan, Z., Luo, P., et al. (2021). ByteTrack: Multi-object tracking by associating every detection box. arXiv preprint arXiv:2110.06864.
- Zhang, Y., Wang, C., Wang, X., Zeng, W., & Liu, W. (2020). FairMOT: On the fairness of detection and re-identification in multiple object tracking.
- Zhou, X., Koltun, V., & Krhenbühl, P. (2020). *Tracking Objects as Points*. arXiv.
- Zhou, X., Wang, D., & Krhenbühl, P. (2019). Objects as points.
- Zhou, T., Ye, Q., Luo, W., Zhang, K., Shi, Z., & Chen, J. (2023). F&F Attack: Adversarial attack against multiple object trackers by inducing false negatives and false positives. In *2023 IEEE/CVF international conference on computer vision*.
- Zhu, J., Yang, H., Liu, N., Kim, M., Zhang, W., & Yang, M. H. (2019). Online multi-object tracking with dual matching attention networks.